

Experiment 3: Water Jug Problem

Aim

To solve the Water Jug Problem using Artificial Intelligence techniques.

Objective

- Understand the problem.
- Apply AI search/backtracking.
- Obtain the required solution.

Algorithm

1. Read jug capacities and target.
2. Fill, empty or pour.
3. Search states.
4. Stop at target.

Program (Python)

```
from collections import deque

def water_jug(jug1, jug2, target):

    visited = set()

    queue = deque([((0, 0), [])])

    while queue:

        (a, b), path = queue.popleft()

        if (a, b) in visited:

            continue

        visited.add((a, b))

        path = path + [(a, b)]

        if a == target or b == target:

            print("Solution Steps:")

            for state in path:

                print(state)

            return

    next_states = [

        (jug1, b),

        (a, jug2),

        (0, b),

        (a, 0),

        (max(0, a - (jug2 - b)), min(jug2, b + a)),

        (min(jug1, a + b), max(0, b - (jug1 - a)))

    ]

    for state in next_states:
```

```

    if state not in visited:
        queue.append((state, path))

    print("No solution exists.")

jug1 = 4
jug2 = 3
target = 2

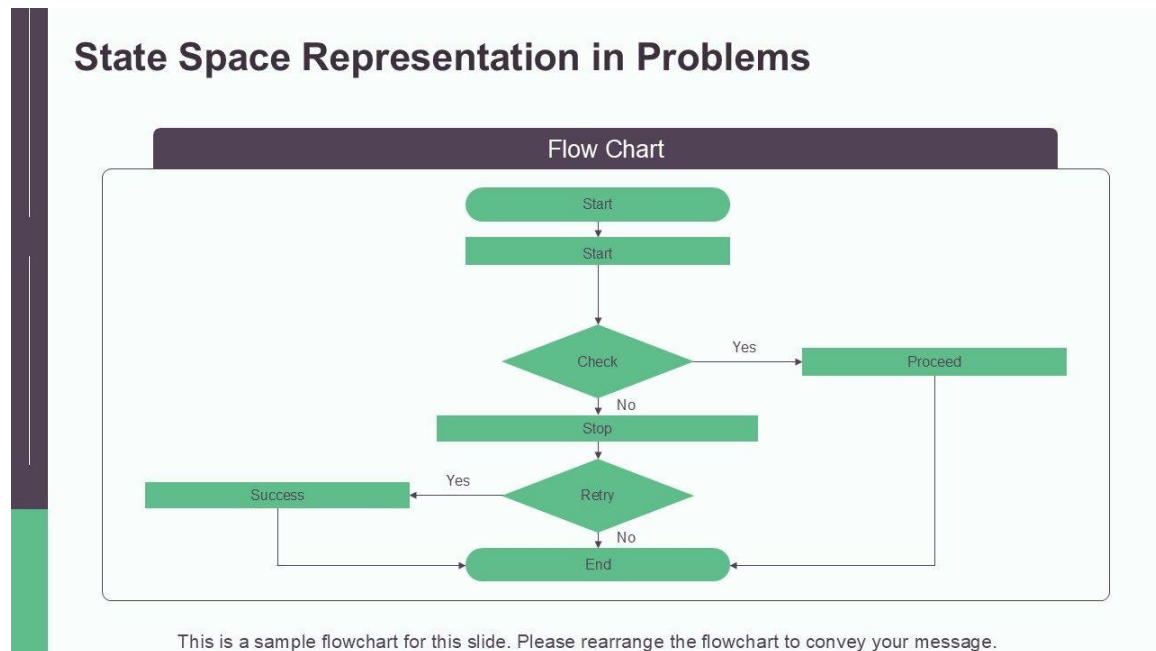
```

```

water_jug(jug1, jug2, target)

```

Flow chart:



Sample Input

Jug1=4 Jug2=3 Target=2

Sample Output

(2,3)

Result

Water Jug solved.

Conclusion

The problem was successfully solved using AI techniques.

```
1 from collections import deque
2 def water_jug(jug1, jug2, target):
3     visited = set()
4     queue = deque([(0, 0), []])
5     while queue:
6         (a, b), path = queue.popleft()
7         if (a, b) in visited:
8             continue
9         visited.add((a, b))
10        path = path + [(a, b)]
11        if a == target or b == target:
12            print("Solution Steps:")
13            for state in path:
14                print(state)
15            return
16        next_states = [
17            (jug1, b),
18            (a, jug2),
19            (0, b),
20            (a, 0),
21            (max(0, a - (jug2 - b)), min(jug2, b + a)),
22            (min(jug1, a + b), max(0, b - (jug1 - a)))
23        ]
24        for state in next_states:
25            if state not in visited:
26                queue.append((state, path))
27    print("No solution exists.")
28 jug1 = 4
29 jug2 = 3
30 target = 2
31
32 water_jug(jug1, jug2, target)
```

> & C:\Users\athav\AppData\Local\

rograms\Python\Python313\python.exe c:/Users/athav/3.py

Solution Steps:

(0, 0)

(0, 3)

(3, 0)

(3, 3)

(4, 2)